



TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: 151287

Need help? Post on our [class slack channel](#) or email us at support@codepath.org

[Getting Started](#)[Overview](#)[Session #1](#)[Session #2](#)[Assignment](#)[Cheatsheet](#)[Learning with AI](#) ✨[Resources](#)[Learning with AI](#)[Report](#)[▶ Course Info](#)[Course Progress](#)[Unit 1](#)[Unit 2](#)[Unit 3](#)[Unit 4](#)[Unit 5](#)[Unit 6](#)[Unit 7](#)

Unit 2: Session 2

Dictionaries in Python II

Students will continue to expand their expertise in Python through the exploration of data structures such as lists and dictionaries. They engage in various tasks like verifying list properties, creating and updating dictionaries, and analyzing data to make decisions.

Through exercises that involve stock management, GPA calculation, and determining the best-rated books or genres, students solidify their understanding of Python's capabilities in handling complex data manipulations and condition-based logic, equipping them for more advanced programming challenges.

You can find all resources from today including the session deck, session recording, and more on the [resources tab](#)



Part 1 : Instructor Led Session

Unit 8

Unit 9

Unit 10

Submission
Guide

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
 - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

► Note on Expectations

Problem Solving Approach

To build a long-term organized approach to problem solving, we'll start with three main steps. We'll refer to them as **UPI: Understand, Plan, and Implement**.

We'll apply these three steps to most of the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem,
- **Plan** a solution step-by-step, and
- **Implement** the solution

- **Comment on UPI**
- **UPI Example**

[Unit 2 Cheatsheet](#)

To jump start your journey into Python dictionaries, we've put together a guide to common functions and syntax you will use throughout Unit 2 breakout problems. Use this cheatsheet as a quick reference guide as you work through the problems below.

Problem Set Version 1

▼ Problem 1: Cast Vote

Write a function `cast_vote()` that records a vote for a candidate in an election. The function accepts a dictionary `votes` that maps candidates to their current number of votes and a string `candidate` that represents the candidate the user would like to vote for. If the candidate doesn't exist, add them to the dictionary. The function should return the updated dictionary.

```
def cast_vote(votes, candidate):  
    pass
```

Example Usage:

```
votes = {"Alice": 5, "Bob": 3}  
cast_vote(votes, "Alice")  
print(votes)  
cast_vote(votes, "Gina")  
print(votes)
```

Example Output:

```
{'Alice': 6, 'Bob': 3}  
{'Alice': 6, 'Bob': 3, 'Gina': 1}
```

► ✨ AI Hint: Accessing Values in a Dictionary

► 💡 Hint: Dictionary Access options

Close Section

▼ Problem 2: Keys in Common

Write a function that takes in two dictionaries, `dict1` and `dict2` and finds all keys common to both dictionaries. The function returns a list of common keys.

```
def common_keys(dict1, dict2):  
    pass
```

Example Usage:

```
dict1 = {"a": 1, "b": 2, "c": 3}  
dict2 = {"b": 4, "c": 5, "d": 6}  
common_list = common_keys(dict1, dict2)  
print(common_list)
```

Example Output:

```
['b', 'c']
```

► 💡 Hint: Accessing Keys, Values, and Key-Value Pairs

► 💡 Hint: Looping over Key-Value Pairs

▼ Problem 3: Frequency Count

Write a function that takes in a list of integers `nums` and counts the number of occurrences of each integer. The function returns the result as a dictionary with integers as keys and their counts as values.

```
def count_occurrences(nums):  
    pass
```

Example Input: `nums = [1, 2, 2, 3, 3, 3, 4]`

Example Output: `{1: 1, 2: 2, 3: 3, 4: 1}`

► ✨ AI Hint: Frequency Maps

▼ Problem 4: Highest Priority Task

Given a dictionary `tasks` where keys are task names and values are priorities (1-10, where 10 is the highest priority), write a function `get_highest_priority_task()` that removes the task with the highest priority from the dictionary and returns its name.

If two tasks have the same priority, return the task that comes first in the alphabet.

```
def get_highest_priority_task(tasks):  
    pass
```

Example Usage:

```
tasks = {"task1": 8, "task2": 10, "task3": 9, "task4": 10, "task5": 7}  
perform_task = (get_highest_priority_task(tasks))  
print(perform_task)
```

```
perform_task = (get_highest_priority_task(tasks))
print(perform_task)

perform_task = (get_highest_priority_task(tasks))
print(perform_task)

print(tasks)
```

Example Output:

```
task2
task4
task3
{'task1': 8, 'task5': 7}
```

►  **Hint: Removing Key-Value Pairs from a Dict**

Close Section

▼ Problem 5: Find Majority Element

Write a function `find_majority_element()` that takes in a list of integers `elements` and finds the majority element in the list. A majority element is an element that appears more than `n/2` times where `n` is the size of the list. If there is no majority element, return `None`.

```
def find_majority_element(elements):
    pass
```

Example Usage:

```
elements = [2, 2, 1, 1, 1, 2, 2]
print(find_majority_element(elements))
```

Example Output: 2

Close Section

▼ Problem 6: Has Duplicates

Write a function `has_duplicates()` that takes in a list of integers `nums` and a positive number `k` as parameters. The function returns `True` if the list contains any duplicate elements within `k` (inclusive) indices of each other. In other words, return `True` if `nums[i]` has the same value as any of the `k` neighboring elements to its left or right. If `k` is greater than the list's length, the solution should check for duplicates in the complete list. The function should return `False` otherwise.

```
def has_duplicates(nums, k):  
    pass
```

Example Usage:

```
nums = [5, 6, 8, 2, 6, 4, 9]  
check1 = has_duplicates(nums, 2)  
print(check1)  
check2 = has_duplicates(nums, 5)  
print(check2)  
check3 = has_duplicates(nums, 3)  
print(check3)
```

Example Output:

```
False  
True  
True
```

Close Section

▼ Problem 7: Make Pairs

Write a function `divide_list()` that takes in an integer list `nums` consisting of `2*n` integers as parameters. The function divides `nums` into `n` pairs such that:

- Each element belongs to exactly one pair
- The elements present in a pair are equal

Return `True` if `nums` can be divided into `n` pairs, otherwise return `False`.

```
def divide_list(nums):  
    pass
```

Example Input:

```
nums = [3,2,3,2,2,2]  
print(divide_list(nums))
```

Example Output: `True`

Explanation: There are 6 elements in `nums`, so they should be divided into $6 / 2 = 3$ pairs. If `nums` is divided into the pairs (2, 2), (3, 3), and (2, 2), it will satisfy all the conditions.

Example Input:

```
nums = [1,2,3,4]  
print(divide_list(nums))
```

Example Output: `False`

Explanation: There is no way to divide `nums` into $4 / 2 = 2$ pairs such that the pairs satisfy every condition.

Close Section

Problem Set Version 2

► Problem 1: Update Score

- ▶ Problem 2: Dictionary Intersection
- ▶ Problem 3: Filter by Price
- ▶ Problem 4: Find Odd Occurrences
- ▶ Problem 5: Find Mode
- ▶ Problem 6: How Many Smaller
- ▶ Problem 7: Good Pairs

Problem Set Version 3

- ▶ Problem 1: Return Book
- ▶ Problem 2: Dictionary Difference
- ▶ Problem 3: Take from Stock
- ▶ Problem 4: Group By Frequency
- ▶ Problem 5: No Duplicates Allowed
- ▶ Problem 6: First Repeating Element
- ▶ Problem 7: Target Sum